IMPLEMENTATION OF RECURRENT NEURAL NETWORK (RNN) FOR TEXT SEQUENCE PROCESSING

Objective: Build an RNN that reads text as an ordered sequence of words and classifies customer reviews as Positive or Negative. The experiment covers tokenization, embeddings, hidden states, backpropagation through time and the vanishing-gradient problem.

1. Execution Workflow

Stage	What happens
Input dataset	Read customer reviews and labels from sentiment_dataset.csv.
Text preprocessing	Convert words into integer token IDs and pad or truncate sequences.
Embedding layer	Map each token ID to a trainable numerical vector.
SimpleRNN layer	Read tokens in order and update a hidden state at each time step.
Output layer	Use sigmoid activation to estimate Positive sentiment probability.
Evaluation	Calculate accuracy, precision, recall and F1-score; inspect confusion matrix and loss curves.

2. Dataset Preparation

Use the previously created **sentiment_dataset.csv**. It contains columns named **Review** and **Sentiment**, with 100 synthetic reviews: 50 Positive and 50 Negative.

Review	Sentiment
The product is excellent	Positive
Amazing quality and fast delivery	Positive
Very poor quality	Negative
The service was terrible	Negative

Dataset note: This small synthetic dataset is suitable for demonstrating execution, but it is not sufficiently large or diverse for reliable real-world performance evaluation.

3. Software Requirements

Use Python with TensorFlow/Keras, pandas, NumPy, scikit-learn, Matplotlib and Seaborn. A CPU is sufficient for this exercise.

```
python -m pip install --upgrade pip
pip install tensorflow pandas numpy scikit-learn matplotlib seaborn
```

Verify TensorFlow installation:

```
python -c "import tensorflow as tf; print(tf.__version__)"
```

4. RNN Architecture and Concepts

An RNN processes a sentence one token at a time. After reading each token, it updates a **hidden state** that carries information from earlier tokens to the next time step.

Example sequence: The | product | is | excellent

Time step 1	Time step 2	Time step 3	Time step 4
The	product	is	excellent
Hidden state h1	Hidden state h2	Hidden state h3	Hidden state h4

Main components

Component	Role
TextVectorization	Builds a vocabulary and converts each review into a fixed-length sequence of token IDs.
Embedding	Learns a dense vector representation for each token ID; this example uses 32 dimensions.
SimpleRNN	Processes the sequence in order and updates the hidden state; the example uses 32 units and tanh activation.
Dense output	One neuron with sigmoid activation produces a probability between 0 and 1.

Hidden-state equation

h(t) = tanh(Wx x(t) + Wh h(t-1) + b)

x(t) is the current token representation; h(t) is the current hidden state; h(t-1) is the previous hidden state; Wx and Wh are weight matrices; b is a bias vector; and tanh is the activation.

Vanishing-gradient problem

During backpropagation through time, gradients are propagated backward across the sequence. For long sequences, repeated multiplication by small derivatives can make gradients very small. Earlier time steps then receive weak learning signals, so a basic RNN may struggle to learn long-range dependencies.

How LSTM helps: LSTM uses gates and a cell state to help retain useful information over longer sequences. It is designed to mitigate the vanishing-gradient problem, not guarantee its elimination.

5. Complete Python Implementation (Part 1)

Create a file named **rnn_text_classification.py** in the same folder as **sentiment_dataset.csv**. Add the following code first:

```
# RNN FOR TEXT SEQUENCE PROCESSING

import numpy as np
import pandas as pd
import tensorflow as tf
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, classification_report
)

# STEP 1: Load dataset
data = pd.read_csv("sentiment_dataset.csv")
X = data["Review"].astype(str).to_numpy()

# Convert Positive = 1 and Negative = 0
y = (
    data["Sentiment"].str.strip().str.lower()
    .map({"negative": 0, "positive": 1})
)

if y.isna().any():
    raise ValueError("Dataset contains invalid sentiment labels.")

y = y.to_numpy(dtype="int32")

# Set random seeds
np.random.seed(42)
tf.random.set_seed(42)

# STEP 2: Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# STEP 3: Convert reviews into integer sequences
MAX_WORDS = 3000
MAX_LENGTH = 30

vectorizer = tf.keras.layers.TextVectorization(
    max_tokens=MAX_WORDS,
    standardize="lower_and_strip_punctuation",
    split="whitespace",
    output_mode="int",
    output_sequence_length=MAX_LENGTH
)

# Learn vocabulary from training data only
vectorizer.adapt(X_train)
vocab_size = len(vectorizer.get_vocabulary())
print("Vocabulary size:", vocab_size)
```

5. Complete Python Implementation (Part 2)

Continue the same Python file:

```
# STEP 4: Build the RNN architecture
model = tf.keras.Sequential([
    # One input string per review
    tf.keras.Input(shape=(), dtype=tf.string),
    vectorizer,

    # Embedding layer: 32-dimensional word vectors
    tf.keras.layers.Embedding(
        input_dim=vocab_size,
        output_dim=32,
        mask_zero=True
    ),

    # Recurrent layer: 32 hidden units
    tf.keras.layers.SimpleRNN(
        units=32,
        activation="tanh",
        return_sequences=False
    ),

    # Binary classification output
    tf.keras.layers.Dense(1, activation="sigmoid")
])

# STEP 5: Compile the model
model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

print("\\nRNN MODEL ARCHITECTURE")
model.summary()

# STEP 6: Train the model
early_stop = tf.keras.callbacks.EarlyStopping(
    monitor="val_loss",
    patience=5,
    restore_best_weights=True
)

print("\\nTraining RNN model...")
history = model.fit(
    X_train,
    y_train,
    epochs=20,
    batch_size=8,
    validation_split=0.2,
    callbacks=[early_stop],
    verbose=1
)

print("\\nTraining completed!")
```

5. Complete Python Implementation (Part 3)

Continue the same Python file:

```
# STEP 7: Evaluate the model
test_loss, test_accuracy = model.evaluate(
    X_test, y_test, verbose=0
)

print("\\nRNN MODEL PERFORMANCE")
print("Test Loss:", round(test_loss, 4))
print("Accuracy:", round(test_accuracy, 4))

# STEP 8: Predict the test data
probabilities = model.predict(X_test).ravel()
predictions = (probabilities >= 0.5).astype(int)

print("Precision:", precision_score(
    y_test, predictions, zero_division=0
))
print("Recall:", recall_score(
    y_test, predictions, zero_division=0
))
print("F1 Score:", f1_score(
    y_test, predictions, zero_division=0
))

print("\\nClassification Report:")
print(classification_report(
    y_test, predictions,
    labels=[0, 1],
    target_names=["Negative", "Positive"],
    zero_division=0
))

# STEP 9: Display confusion matrix
cm = confusion_matrix(y_test, predictions, labels=[0, 1])

plt.figure(figsize=(6, 4))
sns.heatmap(
    cm, annot=True, fmt="d", cmap="Blues",
    xticklabels=["Negative", "Positive"],
    yticklabels=["Negative", "Positive"]
)
plt.title("RNN Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.tight_layout()
plt.show()

# STEP 10: Plot training and validation loss
plt.figure(figsize=(7, 4))
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.title("RNN Training Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

5. Complete Python Implementation (Part 4)

Add the final prediction and model-saving section to the same Python file:

```
# STEP 11: Predict new reviews
new_reviews = np.array([
    "This product is excellent and amazing",
    "The product is very poor and disappointing"
])

new_probabilities = model.predict(new_reviews).ravel()

print("\\nNEW REVIEW PREDICTIONS")
for review, probability in zip(new_reviews, new_probabilities):
    sentiment = "Positive" if probability >= 0.5 else "Negative"
    print("\\nReview:", review)
    print("Positive Probability:", round(float(probability), 4))
    print("Predicted Sentiment:", sentiment)

# STEP 12: Save the trained model
model.save("rnn_sentiment_model.keras")
print("\\nModel saved successfully!")
```

The complete program is now ready to execute.

6. How to Execute the Program

1. Create a folder named **RNN_Text_Sequence**.

2. Place **sentiment_dataset.csv** and **rnn_text_classification.py** inside the folder.

3. Open PowerShell, Command Prompt or the VS Code terminal.

4. Navigate to the project folder using the cd command.

5. Install TensorFlow and the other required libraries.

6. Run the Python file and observe the training and evaluation outputs.

Terminal commands

```
cd path\to\RNN_Text_Sequence
python -m pip install --upgrade pip
pip install tensorflow pandas numpy scikit-learn matplotlib seaborn
python rnn_text_classification.py
```

7. Important RNN Parameters

Parameter	Explanation
MAX_WORDS = 3000	Maximum vocabulary size retained by TextVectorization.
MAX_LENGTH = 30	Each review is padded or truncated to 30 tokens.
Embedding output_dim = 32	Each token is represented by a trainable vector of 32 numbers.
SimpleRNN units = 32	The recurrent layer contains 32 hidden units.
activation = tanh	Applies tanh activation to the recurrent state calculation.
Dense(1, sigmoid)	Produces one output probability for binary classification.
optimizer = adam	Updates model weights during training.
epochs = 20	Maximum passes through the training subset; early stopping may stop sooner.
batch_size = 8	Number of training examples processed before an optimizer update.

8. Understanding the Results

Output	Meaning
Accuracy	Fraction of test reviews classified correctly.
Precision	Of reviews predicted as a class, the fraction that truly belongs to that class.
Recall	Of actual reviews in a class, the fraction correctly identified.
F1-score	Harmonic mean of precision and recall.
Confusion matrix	Counts of correct and incorrect predictions for Negative and Positive classes.
Training loss	Error value on training data as the model learns.
Validation loss	Error value on validation data; useful for observing overfitting.

Expected outputs: Vocabulary size, model architecture, epoch-by-epoch training progress, test metrics, new-review predictions, confusion matrix and training/validation loss graph. The trained model is saved as **rnn_sentiment_model.keras**. Actual scores depend on execution.

9. Vanishing-Gradient Problem

In backpropagation through time, the error signal moves backward through recurrent time steps. When gradients become very small, early time steps receive weak updates. A basic RNN may then struggle to learn dependencies between words that are far apart.

Step	What happens
1	The RNN reads words one after another.
2	Each word updates the hidden state.
3	The network calculates prediction error.
4	Backpropagation through time sends gradients backward across the sequence.
5	Gradients may shrink over many time steps, weakening learning for earlier words.

10. Optional Extension: Replace SimpleRNN with LSTM

Replace only the SimpleRNN layer in the model definition with the following LSTM layer. Keep the Embedding and output layers unchanged, then train and evaluate again.

```
tf.keras.layers.LSTM(
    units=32,
    return_sequences=False
)
```

LSTM uses gates and a cell state to help retain useful information over longer sequences. Compare the RNN and LSTM results, but do not assume LSTM must always achieve higher accuracy on a very small dataset.

11. Suggested Laboratory Experiment

Title: Implementation of Recurrent Neural Network (RNN) for Text Sequence Processing.

Aim: To implement an RNN for text-sequence classification and understand sequential data processing, hidden-state representation and the vanishing-gradient problem.

Learning Outcomes

1. Understand the architecture of an RNN.
2. Explain sequential data processing and hidden-state updates.
3. Implement an RNN using Python and TensorFlow.
4. Understand backpropagation through time and the vanishing-gradient problem.
5. Evaluate an RNN using standard classification metrics.

12. Suggested Teaching Sequence

Order	Demonstration
1	Show how a sentence is tokenized and converted into a sequence of integers.
2	Explain how an embedding layer converts token IDs into vectors.
3	Trace how SimpleRNN updates its hidden state as it reads each token.
4	Run the program and discuss training loss and classification metrics.
5	Explain vanishing gradients and introduce LSTM as an alternative recurrent architecture.

Dataset reminder: The 100-review synthetic dataset is a teaching example. Use a larger, more diverse and independently labelled dataset for meaningful model evaluation or research.

13. Documentation

TensorFlow installation: tensorflow.org/install/pip
Keras TextVectorization: Keras TextVectorization
Keras SimpleRNN: Keras SimpleRNN
Keras LSTM: Keras LSTM